

Manual: Engenharia de Software

Ciclos de Vida (Preditivos e RUP), Metodologias Ágeis (Scrum, Kanban, XP), Engenharia de Requisitos, Modelagem UML, Arquiteturas, Padrões GoF e Testes

CHANCELA EDITORIAL YLUNA85 LABS

Preparação Conjunta: Auditor Fiscal (TI) & Agente de Tributos Estaduais

Revisão Ortográfica e Glossário Geral de Siglas

Material de livre circulação interna. Revisado em Português Brasileiro (pt-BR).

1. Modelos de Ciclo de Vida de Software

O ciclo de vida do software descreve as fases pelas quais o sistema passa, desde a concepção até o descarte. Os modelos de ciclo de vida definem a ordem e o rito de execução dessas fases. Modelos Clássicos (Preditivos) e Iterativos

- **Modelo Cascata (Waterfall):**
Modelo sequencial e rígido. Cada fase (Requisitos, Projeto, Codificação, Testes, Manutenção) deve ser totalmente concluída antes do início da próxima. Dificulta alterações de requisitos ao longo do projeto.
- **Modelo Espiral (Barry Boehm):**
Modelo evolucionário estruturado em ciclos repetitivos (espirais), com foco central na ANALISE DE RISCOS em cada volta da espiral.
- **RUP (Rational Unified Process):**
Processo iterativo orientado a Casos de Uso e centrado na arquitetura. Divide-se em 4 fases sequenciais: 1) Iniciação (Inception - define escopo inicial); 2) Elaboração (Elaboration - projeta a arquitetura e mitiga riscos); 3) Construção (Construction - desenvolve o sistema); 4) Transição (Transition - implantação e testes no cliente).

RUP E AS DISCIPLINAS

Atenção para as provas: no RUP, o trabalho é estruturado em Disciplinas (ex: Requisitos, Análise e Projeto, Implementação, Testes) que ocorrem simultaneamente, com intensidades variadas, ao longo de todas as 4 Fases do projeto.

2. Metodologias Ágeis: Scrum, Kanban e XP

As metodologias ágeis priorizam a adaptabilidade, entregas frequentes de software funcionando e colaboração com o cliente, conforme o Manifesto Ágil de 2001. O Framework Scrum é um framework leve projetado para ajudar equipes a gerar valor por meio de soluções adaptativas para problemas complexos. Seus pilares são: Transparência, Inspeção e Adaptação.

- **Papéis no Scrum:**

1) Product Owner (PO - dono do produto, representa os clientes e define a prioridade do Product Backlog); 2) Scrum Master (SM - líder servidor que garante a aplicação do Scrum e remove impedimentos); 3) Developers (Equipe de Desenvolvimento - constrói o incremento).

- **Eventos do Scrum:**

Sprint (ciclo de desenvolvimento de 1 a 4 semanas); Sprint Planning (planejamento da meta); Daily Scrum (reunião diária de 15 minutos); Sprint Review (demonstração do incremento concluído); Sprint Retrospective (análise de melhoria de processos da equipe).

- **Artefatos do Scrum:**

Product Backlog (lista ordenada de tudo que é necessário no produto); Sprint Backlog (tarefas selecionadas para a Sprint atual); Incremento (soma de itens concluídos na Sprint). Kanban e XP (Extreme Programming)

- **Kanban:**

Método visual focado no gerenciamento do fluxo de trabalho. Utiliza quadros de colunas ('A Fazer', 'Em Execução', 'Concluído') e limita o Trabalho em Progresso (limite de WIP) para evitar gargalos.

- **XP (Extreme Programming):**

Metodologia ágil voltada para engenharia de software com foco em qualidade técnica. Práticas principais: Programação em Par (Pair Programming), Test-Driven Development (TDD), Refatoração e Integração Contínua.

O SCRUM MASTER NÃO É CHEFE

O Scrum Master não é um gerente de projetos tradicional ou chefe da equipe. Ele é um facilitador e líder servidor. Não atribui tarefas aos desenvolvedores; a equipe de desenvolvimento é auto-organizável e define como executar o trabalho.

3. Engenharia de Requisitos

A engenharia de requisitos engloba as atividades de descoberta, análise, especificação e gerenciamento das necessidades dos usuários do sistema. Classificação dos Requisitos

- **Requisitos Funcionais (RF):**
Descrevem as funções e serviços que o sistema deve fornecer (o que o sistema deve fazer). Ex: 'O sistema deve permitir a emissão de Nota Fiscal Eletrônica'.
- **Requisitos Não Funcionais (RNF):**
Descrevem as restrições, qualidades ou propriedades sobre os serviços fornecidos (como o sistema deve fazer). Abrangem desempenho, segurança, usabilidade, confiabilidade e portabilidade. Ex: 'O sistema deve responder as consultas em menos de 2 segundos' (Desempenho).
- **Regras de Negócio (RN):**
Diretrizes, normas ou políticas da organização que determinam como as ações ocorrem. Não são requisitos de software diretos, mas guiam a criação deles. Ex: 'O desconto máximo para clientes VIP é de 15%'. Processo de Requisitos
- **Elicitação e Análise:**
Descoberta de requisitos através de entrevistas, brainstorming, prototipação e observação.
- **Especificação:**
Documentação dos requisitos em formato legível (Documento de Requisitos, Histórias de Usuário ou Casos de Uso).
- **Validação:**
Verificação com o cliente de que os requisitos especificados são realmente o que eles desejam.

HISTÓRIAS DE USUÁRIO (USER STORIES)

Em metodologias ágeis, os requisitos são documentados como Histórias de Usuário sob o formato: 'Como um [ator], eu quero [ação], para que eu possa [benefício]'. São acompanhadas por Critérios de Aceitação.

4. Modelagem UML: Diagramas Estruturais

A UML (Unified Modeling Language) é a notação padrão para modelagem orientada a objetos de sistemas de software, dividida em diagramas estruturais e comportamentais. Diagrama de Classes (O mais Cobrado) Representa a estrutura estática do sistema, mostrando as classes, seus atributos, métodos e relacionamentos:

- **Associação Simples:**
Relacionamento básico de conexão estrutural entre duas classes. Ex: Cliente compra Produto.
- **Agregação (Relação Todo-Parte fraca - Losango Vazio):**
Indica que a classe parte pode existir independentemente da classe todo. Ex: Computador possui Monitores externos (se o computador for destruído, os monitores continuam existindo).
- **Composição (Relação Todo-Parte forte - Losango Preenchido):**
Indica dependência física ou existencial da parte em relação ao todo. Se o todo for destruído, as partes também são destruídas. Ex: Nota Fiscal possui Itens da Nota Fiscal.
- **Generalização / Especialização (Herança - Seta Vazia):**
Indica herança de atributos e métodos de uma superclasse por uma subclasse. Ex: Pessoa Física herda de Pessoa.

AGREGAÇÃO VS. COMPOSIÇÃO NO DIAGRAMA

O losango de Agregação é sempre VAZIO (transparente) e indica independência. O losango de Composição é sempre PREENCHIDO (preto) e indica que a parte não tem vida própria sem o todo. Não erre essa representação visual na prova!

5. Modelagem UML: Diagramas Comportamentais

Os diagramas comportamentais mostram a dinâmica do sistema, a interação entre os objetos e a mudança de estado ao longo do tempo. Diagrama de Casos de Uso Mostra as interações entre Atores (usuários ou sistemas externos) e os Casos de Uso (funções do sistema). Os relacionamentos principais entre Casos de Uso são:

- **Include (Inclusão - Seta pontilhada com <<include>>):**

Indica que o caso de uso de origem exige obrigatoriamente a execução do caso de uso de destino. Ex: 'Sacar Dinheiro' inclui 'Validar Senha'.

- **Extend (Extensão - Seta pontilhada com <<extend>>):**

Indica comportamento opcional ou condicional que estende o caso de uso de origem. Ex: 'Efetuar Pagamento' é estendido por 'Exibir Alerta de Saldo Baixo' (condicionalmente). Diagrama de Sequência Diagrama de interação focado no envio de mensagens ordenadas temporalmente entre linhas de vida (objetos) para realizar uma operação. Diagrama de Atividades Representa o fluxo de controle de um processo de negócio ou algoritmo, assemelhando-se a um fluxograma.

DIREÇÃO DA SETA DE RELACIONAMENTOS

No relacionamento <<include>>, a seta aponta do caso de uso base para o caso incluído. No relacionamento <<extend>>, a seta aponta do caso extensor (opcional) para o caso de uso base. Essa direção é alvo constante de pegadinhas de bancas.

6. Arquiteturas de Software: MVC, Microsserviços e SOA

A arquitetura de software define a estrutura organizacional do sistema, seus componentes, relacionamentos e princípios de design. MVC (Model-View-Controller) Padrão de arquitetura que divide a aplicação em três camadas interconectadas para separar a representação da informação do controle do usuário:

- **Model (Modelo):**
Gere os dados, as regras de negócio, a persistência e o estado do sistema.
- **View (Visão):**
Representação visual dos dados para o usuário (interface grafica, telas HTML/CSS).
- **Controller (Controlador):**
Intercepta a entrada do usuário na View, converte as ações em comandos para o Model e atualiza a View correspondente. Microsserviços vs. Monolítico
- **Monolítico:**
O sistema é construído como uma única unidade autônoma de implantação. Facil de testar inicialmente, mas complexo de escalar e manter a medida que cresce.
- **Microsserviços:**
O sistema é decomposto em um conjunto de serviços pequenos, independentes, autônomos e altamente focados. Cada serviço roda seu próprio processo e se comunica por APIs (REST). Facilita escalabilidade e tolerância a falhas. SOA - SERVICE-ORIENTED ARCHITECTURE SOA é um estilo de arquitetura de software baseado no fornecimento de serviços de negócio reutilizáveis. Diferente de microsserviços (que visam desacoplamento físico máximo de implantação), SOA foca no compartilhamento e integração de sistemas legados através de um barramento de serviços (ESB).

7. Padrões de Projeto GoF (Gang of Four)

Os padrões de projeto são soluções consolidadas para problemas comuns recorrentes no desenvolvimento orientados a objetos. Dividem-se em três categorias principais: 1. Padrões Criacionais (Criação de Objetos)

- **Singleton:**
Garante que uma classe tenha apenas uma única instância na memória de toda a aplicação e fornece um ponto de acesso global a ela.
- **Factory Method / Abstract Factory:**
Factory Method define uma interface para criar um objeto, mas deixa as subclasses decidirem qual classe instanciar. Abstract Factory cria famílias de objetos relacionados sem especificar suas classes concretas.
- **Adapter:**
Converte a interface de uma classe em outra interface esperada pelo cliente, permitindo a cooperação de interfaces incompatíveis. Ex: plugues e tomadas de padrões diferentes.
- **Façade (Fachada):**
Fornece uma interface simplificada para um subsistema complexo.
- **Decorator (Decorador):**
Adiciona responsabilidades extras a um objeto de forma dinâmica (em tempo de execução) sem usar herança.
- **Observer (Observador):**
Define uma dependência um-para-muitos, notificando automaticamente todos os observadores de qualquer mudança de estado no objeto observado.
- **Strategy (Estratégia):**
Define uma família de algoritmos, encapsula cada um e os torna intercambiáveis em tempo de execução.

STRATEGY VS. STATE

Ambos possuem diagramas UML parecidos, mas objetivos distintos: o Strategy permite alterar algoritmos intercambiáveis de forma explícita pelo cliente. O State permite alterar o comportamento de um objeto conforme o seu estado interno muda de forma transparente.

8. Qualidade e Testes de Software: Caixa Preta vs. Branca

Os testes de software garantem que o sistema atenda aos requisitos especificados e esteja livre de falhas críticas. Níveis de Testes

- **Teste Unitário (de Unidade):**
Testa a menor unidade isolada de código (funções, metodos ou classes). Executado pelos proprios desenvolvedores.
- **Teste de Integração:**
Avalia a interação e comunicação entre diferentes modulos ou unidades integradas do sistema.
- **Teste de Sistema (ou de Sistema Completo):**
Avalia o comportamento global do sistema em relação aos requisitos funcionais e não funcionais especificados.
- **Teste de Aceitacao (UAT):**
Validação final realizada pelos usuários finais para verificar se o sistema atende as suas necessidades de negócio antes de ir para produção. Técnicas de Testes de Software
- **Caixa Preta (Funcional):**
O testador não conhece o código fonte interno. Os testes sao focados nas entradas e saídas em relação a especificação funcional do sistema. Ex: teste de interface de login.
- **Caixa Branca (Estrutural):**
O testador conhece o código fonte interno. Os testes avaliam caminhos lógicos, condicionais e estruturas internas do código. Ex: garantir que todas as linhas de um algoritmo 'IF/ELSE' sejam executadas (cobertura de código). CI/CD - INTEGRAÇÃO E IMPLANTAÇÃO CONTINUAS A Integração Contínua (CI) é a prática de integrar mudanças de código ao repositório principal com frequência, rodando testes automáticos. A Implantação Contínua (CD) automatiza o deploy seguro dessas mudanças nos ambientes de produção.

9. As 10 Grandes Pegadinhas de Engenharia de Software

As bancas FGV e Cebraspe exploram pegadinhas conceituais sobre engenharia de software de forma frequente. Atente-se aos seguintes pontos:

- **1. Cascata não permite retorno:**
No modelo cascata classico, não ha caminhos de retorno (feedback loops) entre as fases. Uma fase so começa se a anterior estiver 100% pronta.
- **2. Fases do RUP vs. Disciplinas:**
As fases do RUP sao Iniciação, Elaboracao, Construcao e Transicao. Disciplinas sao Requisitos, Análise, etc. As bancas trocam os nomes de fases por disciplinas.
- **3. Product Owner define prioridade, não o Scrum Master:**
O PO é o único responsável por priorizar e ordenar os itens do Product Backlog. O SM apenas facilita os ritos e remove impedimentos.
- **4. Critério de Aceitacao não e Historia de Usuário:**
As Histórias de Usuário definem o requisito do ponto de vista do ator. Os Critérios de Aceitacao definem o que deve ser validado para a historia ser considerada concluida (Definition of Done).
- **5. Agregação vs. Composição no UML:**
Composição (losango preto) indica dependência vital da parte. Agregação (losango vazio) indica ciclo de vida independente. A banca adora trocar os losangos no Diagrama de Classes.
- **6. Include e Obrigatório, Extend e Condicional:**
No UML de Casos de Uso, include e de execução obrigatória do caso secundario. O extend e de execução condicional/opcional.
- **7. Factory Method vs. Abstract Factory:**
Factory Method usa herança (um método criador em subclasse). Abstract Factory usa composição (objeto fabricante contendo metodos para fabricar familias de objetos).
- **8. Decorator não usa Herança estrutural:**
O Decorator estende o comportamento de forma dinâmica através de encapsulamento de objetos, evitando a criação de subclasses estaticas complexas via herança.
- **9. Caixa Preta não avalia Código Fonte:**
Questões que dizem que testes de caixa preta garantem a cobertura de caminhos do código estão incorretas. Caixa preta avalia apenas entradas e saídas funcionais.
- **10. Risco de TDD:**
O Test-Driven Development (TDD) exige escrever o caso de teste ANTES de escrever o código de produção correspondente (Ciclo Red-Green-Refactor).

FICHA DE REVISÃO GERAL

Foque os estudos finais nos ritos e papéis do Scrum (PO, SM, Devs), nos relacionamentos de include/extend em Casos de Uso, nos padrões GoF (Singleton, Factory, Strategy) e nos conceitos de testes (Caixa Preta vs. Branca).

GLOSSÁRIO DE SIGLAS E ABREVIATURAS

Consulte abaixo o significado e a expansão explicativa de todas as siglas contábeis, tributárias e de TI presentes nas apostilas de preparação:

ACID	Atomicidade, Consistência, Isolamento e Durabilidade. Propriedades fundamentais para transações em SGBDs.
BI	Business Intelligence. Inteligência de Negócios para mineração e análise estratégica de dados.
CAP	Consistência, Disponibilidade e Tolerância a Partições. Teorema limitador para sistemas distribuídos.
CEBRASPE	Centro de Seleção e de Promoção de Eventos. Banca examinadora oficial de concursos públicos.
CFC	Conselho Federal de Contabilidade. Órgão responsável pela edição das NBCs no território nacional.
CIAP	Controle de Crédito de ICMS do Ativo Permanente. Controle fiscal de apropriação de créditos de bens imobilizados.
CID	Confidencialidade, Integridade e Disponibilidade. Pilares de sustentação da Segurança da Informação.
COBIT	Control Objectives for Information and Related Technologies. Framework global de Governança de TI.
CPC	Comitê de Pronunciamentos Contábeis. Emissor de normas convergentes com padrões internacionais.
FGV	Fundação Getulio Vargas. Banca examinadora de certames e instituição de pesquisa econômica.
ICMS	Imposto sobre Operações Relativas à Circulação de Mercadorias e Prestações de Serviços de Transporte.
IPVA	Imposto sobre a Propriedade de Veículos Automotores. Tributo de competência estadual.
ITD	Imposto sobre Transmissão Causa Mortis e Doação. Tributo sobre heranças e doações na Bahia.
ITIL	Information Technology Infrastructure Library. Boas práticas de Gerenciamento de Serviços de TI.
KDD	Knowledge Discovery in Databases. Processo sistemático de descoberta de conhecimento em bases de dados.
LDO	Lei de Diretrizes Orçamentárias. Define as metas e prioridades para o orçamento do exercício seguinte.
LOA	Lei Orçamentária Anual. Estima as receitas e fixa as despesas públicas para o exercício financeiro.
NBC TA	Normas Brasileiras de Contabilidade Técnicas de Auditoria Independente. Alinhadas ao padrão internacional.
NBC TI	Normas Brasileiras de Contabilidade Técnicas de Auditoria Interna. Foco nos processos organizacionais.
NoSQL	Not Only SQL. Família de SGBDs não relacionais estruturados para alto desempenho e escalabilidade.
OLAP	Online Analytical Processing. Ferramentas analíticas baseadas em cubos de dados multidimensionais.
PAF-BA	Processo Administrativo Fiscal do Estado da Bahia. Lei nº 3.956/1981 que regula o contencioso.
PPA	Plano Plurianual. Planejamento estratégico governamental de médio prazo (4 anos).
RICMS-BA	Regulamento do ICMS do Estado da Bahia. Decreto regulamentador do principal imposto estadual.
RUP	Rational Unified Process. Processo de desenvolvimento de software disciplinado e preditivo.
SEFAZ-BA	Secretaria da Fazenda do Estado da Bahia. Órgão de arrecadação e controle fiscal baiano.

SGBD	Sistema de Gerenciamento de Banco de Dados. Software de controle e armazenamento seguro de dados.
SQL	Structured Query Language. Linguagem declarativa padrão para consulta em SGBDs relacionais.
TI	Tecnologia da Informação. Recursos de hardware, software e infraestrutura de rede corporativa.
XP	Extreme Programming. Metodologia ágil de engenharia de software focada em excelência técnica.